

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

- INTRODUCCION A LA SINCRONIZACION EN SISTEMAS DISTRIBUIDOS
- SINCRONIZACION DE RELOJES
- RELOJES LOGICOS
- RELOJES FISICOS
- ALGORITMOS PARA LA SINCRONIZACION DE RELOJES
- EXCLUSION MUTUA
- ALGORITMOS DE ELECCION
- TRANSACCIONES ATOMICAS
- EL MODELO DE TRANSACCION
- IMPLANTACION DEL MODELO DE TRANSACCION
- CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION
- BLOQUEOS EN SISTEMAS DISTRIBUIDOS
- DETECCION DISTRIBUIDA DE BLOQUEOS
- PREVENCIÓN DISTRIBUIDA DE BLOQUEOS

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

1

INTRODUCCION A LA SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

- OTRO PROBLEMA DE GRAN IMPORTANCIA ES EL **TIEMPO** Y LA **FORMA DE MEDIRLO**:
 - ◆ JUEGA UN PAPEL FUNDAMENTAL EN ALGUNOS MODELOS DE **SINCRONIZACION**.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

4

INTRODUCCION A LA SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

2

SINCRONIZACION DE RELOJES

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

5

INTRODUCCION A LA SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

- ADEMÁS DE LA **COMUNICACION**, ES FUNDAMENTAL LA **FORMA** EN QUE LOS PROCESOS:
 - ◆ **COOPERAN**.
 - ◆ SE **SINCRONIZAN** ENTRE SI.
- EJEMPLOS:
 - ◆ FORMA DE IMPLANTAR LAS **REGIONES CRITICAS**.
 - ◆ FORMA DE **ASIGNAR RECURSOS** EN UN SISTEMA DISTRIBUIDO.
- LOS PROBLEMAS RELATIVOS A LAS **REGIONES CRITICAS**, **EXCLUSION MUTUA** Y LA **SINCRONIZACION**:
 - ◆ GENERALMENTE SE RESUELVEN EN SISTEMAS DE UNA SOLA CPU CON METODOS COMO LOS SEMAFOROS Y LOS MONITORES:
 - SE BASAN EN LA MEMORIA COMPARTIDA:
 - **NO SON APLICABLES** A SISTEMAS DISTRIBUIDOS.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

3

SINCRONIZACION DE RELOJES

- GENERALMENTE LOS **ALGORITMOS DISTRIBUIDOS** TIENEN LAS SIGUIENTES **PROPIEDADES**:
 - ◆ LA **INFORMACION RELEVANTE** SE **DISTRIBUYE** ENTRE VARIAS MAQUINAS.
 - ◆ LOS **PROCESOS TOMAN LAS DECISIONES** SOLO CON BASE EN LA INFORMACION DISPONIBLE EN FORMA **LOCAL**.
 - ◆ DEBE **EVITARSE UN UNICO PUNTO DE FALLO** EN EL SISTEMA.
 - ◆ **NO EXISTE UN RELOJ COMUN** O ALGUNA OTRA FUENTE PRECISA DEL **TIEMPO GLOBAL**.
- LOS PRIMEROS TRES PUNTOS INDICAN QUE ES **INACEPTABLE REUNIR TODA LA INFORMACION** EN UN SOLO LUGAR PARA SU PROCESAMIENTO:
 - ◆ **LOGRAR LA SINCRONIZACION SIN CENTRALIZACION** REQUIERE HACER LAS COSAS DISTINTAS AL CASO DE LOS **SISTEMAS OPERATIVOS TRADICIONALES**.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

6

SINCRONIZACION DE RELOJES

- EL ULTIMO PUNTO TAMBIEN ES CRUCIAL:
 - ◆ EN UN SISTEMA CENTRALIZADO EL TIEMPO NO ES AMBIGUO.
 - ◆ EN UN SISTEMA DISTRIBUIDO NO ES TRIVIAL PONER DE ACUERDO A TODAS LAS MAQUINAS EN LA HORA.
 - ◆ SE REQUIERE UN ACUERDO GLOBAL EN EL TIEMPO:
 - LA FALTA DE SINCRONIZACION EN LOS RELOJES PUEDE SER DRASTICA EN PROCESOS DEPENDIENTES DEL TIEMPO.
- LA PREGUNTA ES SI ES POSIBLE SINCRONIZAR TODOS LOS RELOJES EN UN SISTEMA DISTRIBUIDO.

RELOJES LOGICOS

- ◆ CUANDO EL CONTADOR LLEGA A "0":
 - SE GENERA UNA INTERRUPCION.
 - EL CONTADOR SE VUELVE A CARGAR MEDIANTE EL R. MANTENEDOR.
- ◆ SE PUEDE PROGRAMAR UN CRONOMETRO PARA QUE GENERE UNA INTERRUPCION "X" VECES POR SEGUNDO.
- ◆ CADA INTERRUPCION SE DENOMINA MARCA DE RELOJ.
- PARA UNA COMPUTADORA Y UN RELOJ:
 - ◆ NO INTERESAN PEQUEÑOS DESFASAJES DEL RELOJ PORQUE:
 - TODOS LOS PROCESOS DE LA MAQUINA USAN EL MISMO RELOJ Y TENDRAN CONSISTENCIA INTERNA.
 - IMPORTAN LOS TIEMPOS RELATIVOS.
- PARA VARIAS COMPUTADORAS CON SUS RESPECTIVOS RELOJES:
 - ◆ ES IMPOSIBLE GARANTIZAR QUE LOS CRISTALES DE COMPUTADORAS DISTINTAS OSCILEN CON LA MISMA FRECUENCIA.
 - ◆ HABRA UNA PERDIDA DE SINCRONIA EN LOS RELOJES (DE SOFTWARE):
 - TENDRAN VALORES DISTINTOS AL SER LEIDOS.

RELOJES LOGICOS

RELOJES LOGICOS

- LA DIFERENCIA ENTRE LOS VALORES DEL TIEMPO SE LLAMA DISTORSION DEL RELOJ:
 - ◆ PODRIA GENERAR FALLAS EN LOS PROGRAMAS DEPENDIENTES DEL TIEMPO.
- LAMPORT DEMOSTRO QUE LA SINCRONIZACION DE RELOJES ES POSIBLE Y PRESENTO UN ALGORITMO PARA LOGRARLO.
- LAMPORT SEÑALO QUE LA SINCRONIZACION DE RELOJES NO TIENE QUE SER ABSOLUTA:
 - ◆ SI 2 PROCESOS NO INTERACTUAN NO ES NECESARIO QUE SUS RELOJES ESTEN SINCRONIZADOS.
 - ◆ GENERALMENTE LO IMPORTANTE NO ES QUE LOS PROCESOS ESTEN DE ACUERDO EN LA HORA:
 - SI IMPORTA QUE COINCIDAN EN EL ORDEN EN QUE OCURREN LOS EVENTOS.

RELOJES LOGICOS

- LAS COMPUTADORAS POSEEN UN CIRCUITO PARA EL REGISTRO DEL TIEMPO CONOCIDO COMO DISPOSITIVO RELOJ.
- ES UN CRONOMETRO CONSISTENTE EN UN CRISTAL DE CUARZO DE PRECISION SOMETIDO A UNA TENSION ELECTRICA QUE:
 - ◆ OSCILA CON UNA FRECUENCIA BIEN DEFINIDA QUE DEPENDE DE:
 - LA FORMA EN QUE SE CORTE EL CRISTAL.
 - EL TIPO DE CRISTAL.
 - LA MAGNITUD DE LA TENSION.
 - ◆ A CADA CRISTAL SE LE ASOCIAN DOS REGISTROS:
 - R. CONTADOR.
 - R. MANTENEDOR.
 - ◆ CADA OSCILACION DEL CRISTAL DECREMENTA EN 1 AL CONTADOR.

RELOJES LOGICOS

- PARA CIERTOS ALGORITMOS LO QUE IMPORTA ES LA CONSISTENCIA INTERNA DE LOS RELOJES:
 - ◆ NO INTERESA SU CERCANIA PARTICULAR AL TIEMPO REAL (OFICIAL).
 - ◆ LOS RELOJES SE DENOMINAN RELOJES LOGICOS.
- LOS RELOJES FISICOS SON RELOJES QUE:
 - ◆ DEBEN SER IGUALES (ESTAR SINCRONIZADOS).
 - ◆ NO DEBEN DESVIARSE DEL TIEMPO REAL MAS ALLA DE CIERTA MAGNITUD.

RELOJES LOGICOS

- PARA SINCRONIZAR LOS RELOJES LOGICOS, LAMPORIT DEFINIO LA RELACION OCURRE ANTES DE (HAPPENS-BEFORE):
 - ◆ SI a Y b SON EVENTOS EN EL MISMO PROCESO Y a OCURRE ANTES DE b , ENTONCES $a \rightarrow b$ ES VERDADERO.
 - ◆ "OCURRE ANTES DE" ES UNA RELACION TRANSITIVA:
 - ⇒ SI $a \rightarrow b$ Y $b \rightarrow c$, ENTONCES $a \rightarrow c$.
 - ◆ SI DOS EVENTOS x E y ESTAN EN PROCESOS DIFERENTES QUE NO INTERCAMBIAN MENSAJES, ENTONCES $x \rightarrow y$ NO ES VERDADERO, PERO TAMPOCO LO ES $y \rightarrow x$:
 - ⇒ SE DICE QUE SON EVENTOS CONCURRENTES.

RELOJES LOGICOS

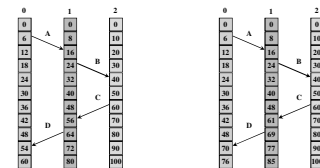
- ◆ LAMPORIT UTILIZA LA RELACION "OCURRE ANTES DE":
 - ⇒ SI "C" SALE EN "60" DEBE LLEGAR EN "61" O EN UN TIEMPO POSTERIOR.
 - ⇒ CADA MENSAJE ACARREA EL TIEMPO DE ENVIO, DE ACUERDO CON EL RELOJ DEL EMISOR.
 - ⇒ CUANDO UN MENSAJE LLEGA Y EL RELOJ DEL RECEPTOR MUESTRA UN VALOR ANTERIOR AL TIEMPO EN QUE SE ENVIO EL MENSAJE:
 - EL RECEPTOR ADELANTA SU RELOJ PARA QUE TENGA UNA UNIDAD MAS QUE EL TIEMPO DE ENVIO.

RELOJES LOGICOS

- NECESITAMOS UNA FORMA DE MEDIR EL TIEMPO TAL QUE A CADA EVENTO a , LE PODAMOS ASOCIAR UN VALOR DEL TIEMPO $C(a)$ EN EL QUE TODOS LOS PROCESOS ESTEN DE ACUERDO:
 - ◆ SE DEBE CUMPLIR QUE:
 - ⇒ SI $a \rightarrow b$ ENTONCES $C(a) < C(b)$.
 - ⇒ EL TIEMPO DEL RELOJ, C , SIEMPRE DEBE IR HACIA ADELANTE (CRECIENTE), Y NUNCA HACIA ATRAS (DECRECIENTE).
 - EL ALGORITMO DE LAMPORIT ASIGNA TIEMPOS A LOS EVENTOS.

RELOJES LOGICOS

EJEMPLO DE TRES PROCESOS CUYOS RELOJES CORREN A DIFERENTES VELOCIDADES - EL ALGORITMO DE LAMPORIT CORREGIÓ LOS RELOJES



RELOJES LOGICOS

- CONSIDERAMOS TRES PROCESOS QUE SE EJECUTAN EN DIFERENTES MAQUINAS, C / U CON SU PROPIO RELOJ Y VELOCIDAD:
 - ◆ EL PROCESO "0" ENVIA EL MENSAJE "A" AL PROCESO "1" CUANDO EL RELOJ DE "0" MARCA "6".
 - ◆ EL PROCESO "1" RECIBE EL MENSAJE "A" CUANDO SU RELOJ MARCA "16".
 - ◆ SI EL MENSAJE ACARREA EL TIEMPO DE INICIO "6", EL PROCESO "1" CONSIDERARA QUE TARDO 10 MARCAS DE RELOJ EN VIAJAR.
 - ◆ EL MENSAJE "B" DE "1" A "2" TARDA 16 MARCAS DE RELOJ.
 - ◆ EL MENSAJE "C" DE "2" A "1" SALE EN "60" Y LLEGA EN "56", TARDARIA UN TIEMPO NEGATIVO, LO CUAL ES IMPOSIBLE.
 - ◆ EL MENSAJE "D" DE "1" A "0" SALE EN "64" Y LLEGA EN "54".

RELOJES LOGICOS

- ESTE ALGORITMO CUMPLE NUESTRAS NECESIDADES PARA EL TIEMPO GLOBAL, SI SE HACE EL SIGUIENTE AGREGADO:
 - ◆ ENTRE DOS EVENTOS CUALESQUIERA, EL RELOJ DEBE MARCAR AL MENOS UNA VEZ.
 - ◆ DOS EVENTOS NO DEBEN OCURRIR EXACTAMENTE AL MISMO TIEMPO.
- CON ESTE ALGORITMO SE PUEDE ASIGNAR UN TIEMPO A TODOS LOS EVENTOS EN UN SISTEMA DISTRIBUIDO, CON LAS SIGUIENTES CONDICIONES:
 - ◆ SI a OCURRE ANTES DE b EN EL MISMO PROCESO, $C(a) < C(b)$.
 - ◆ SI a Y b SON EL ENVIO Y RECEPCION DE UN MENSAJE, $C(a) < C(b)$.
 - ◆ PARA TODOS LOS EVENTOS a Y b , $C(a)$ ES DISTINTO DE $C(b)$.

RELOJES FISICOS

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

19

RELOJES FISICOS

- LOS FISICOS DEFINIERON AL SEGUNDO COMO EL TIEMPO QUE TARDA EL ATOMO DE CESIO 133 PARA HACER 9.192.631.770 TRANSICIONES:
 - ◆ UN SEGUNDO ES LA DURACIÓN DE 9.192.631.770 PERIODOS DE LA RADIACIÓN EMITIDA EN LA TRANSICIÓN ENTRE LOS DOS NIVELES HIPERFINOS DEL ESTADO FUNDAMENTAL DEL ÁTOMO DE CESIO 133, A NIVEL DEL MAR (CON CAMPO MAGNÉTICO CERO).
 - ◆ SE TOMO ESTE N° PARA QUE EL SEGUNDO ATOMICO COINCIDA CON EL SEGUNDO SOLAR PROMEDIO DE 1958.
- LA OFICINA INTERNACIONAL DE LA HORA EN PARIS (BIH) RECIBE LAS INDICACIONES DE CERCA DE 50 RELOJES ATOMICOS EN EL MUNDO Y CALCULA EL TIEMPO ATOMICO INTERNACIONAL (TAI):
 - ◆ BIH: BUREAU INTERNATIONAL DE L'HEURE (INTERNATIONAL TIME BUREAU).

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

22

RELOJES FISICOS

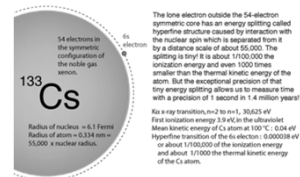
- EL ALGORITMO DE LAMPORT PROPORCIONA UN ORDEN DE EVENTOS SIN AMBIGÜIDADES, PERO:
 - ◆ LOS VALORES DE TIEMPO ASIGNADOS A LOS EVENTOS NO TIENEN PORQUE SER CERCANOS A LOS TIEMPOS REALES EN LOS QUE OCURREN.
 - ◆ EN CIERTOS SISTEMAS (EJ.: SIST. DE TIEMPO REAL), ES IMPORTANTE LA HORA REAL DEL RELOJ:
 - ⇒ SE PRECISAN RELOJES FISICOS EXTERNOS (MAS DE UNO).
 - ⇒ SE DEBEN SINCRONIZAR:
 - CON LOS RELOJES DEL MUNDO REAL.
 - ENTRE SI.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

20

RELOJES FISICOS

- RELOJES ATÓMICOS DE CESIO



SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

23

RELOJES FISICOS

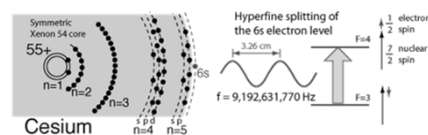
- LA MEDICION DEL TIEMPO REAL CON ALTA PRECISION NO ES SENCILLA.
- DESDE ANTIGUO EL TIEMPO SE HA MEDIDO ASTRONOMICAMENTE.
- SE CONSIDERA EL DIA SOLAR AL INTERVALO ENTRE DOS TRANSITOS CONSECUTIVOS DEL SOL:
 - ◆ EL TRANSITO DEL SOL ES EL EVENTO EN QUE EL SOL ALCANZA SU PUNTO APARENTEMENTE MAS ALTO EN EL CIELO.
- EL SEGUNDO SOLAR SE DEFINE COMO 1 / 86400 DE UN DIA SOLAR.
- COMO EL PERIODO DE ROTACION DE LA TIERRA NO ES CONSTANTE, SE CONSIDERA EL SEGUNDO SOLAR PROMEDIO DE UN GRAN N° DE DIAS.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

21

RELOJES FISICOS

- RELOJES ATÓMICOS DE CESIO

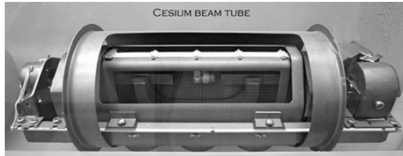


SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

24

RELOJES FISICOS

■ RELOJES ATÓMICOS DE CESIO

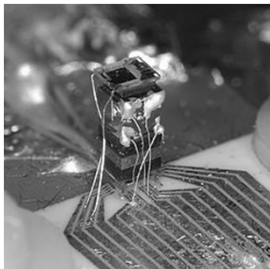


RELOJES FISICOS

- EL INSTITUTO NACIONAL DEL TIEMPO ESTANDAR (NIST) DE EE.UU. Y OTROS DE OTROS PAISES:
 - ◆ OPERAN ESTACIONES DE RADIO DE ONDA CORTA O SATELITES DE COMUNICACIONES.
 - ◆ TRANSMITEN PULSOS UTC CON CIERTA REGULARIDAD ESTABLECIDA (CADA SEGUNDO, CADA 0,5 MSEG, ETC.).
 - ◆ SE DEBEN CONOCER CON PRECISION LA POSICION RELATIVA DEL EMISOR Y DEL RECEPTOR:
 - SE DEBE COMPENSAR EL RETRASO DE PROPAGACION DE LA SEÑAL.
 - SI LA SEÑAL SE RECIBE POR MODEM TAMBIEN SE DEBE COMPENSAR POR LA RUTA DE LA SEÑAL Y LA VELOCIDAD DEL MODEM.
 - SE DIFICULTA LA OBTENCION DEL TIEMPO CON UNA PRECISION EXTREMADAMENTE ALTA.

RELOJES FISICOS

■ RELOJ ATÓMICO EN UN CHIP (NIST)



ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

RELOJES FISICOS

- COMO CONSECUENCIA DE QUE EL DIA SOLAR PROMEDIO ES CADA VEZ MAYOR, UN DIA TAI ES 3 MSEG MENOR QUE UN DSP:
 - ◆ LA BIH INTRODUCE SEGUNDOS DE SALTO PARA HACER LAS CORRECCIONES NECESARIAS PARA QUE PERMANEZCAN EN FASE:
 - EL SISTEMA DE TIEMPO BASADO EN LOS SEGUNDOS TAI.
 - EL MOVIMIENTO APARENTE DEL SOL.
 - ◆ SURGE EL TIEMPO COORDENADO UNIVERSAL (UTC).

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- SI UNA MAQUINA TIENE UN RECEPTOR DE UTC:
 - ◆ TODAS LAS MAQUINAS DEBEN SINCRONIZARSE CON ELLA.
- SI NINGUNA MAQUINA TIENE UN RECEPTOR DE UTC:
 - ◆ CADA MAQUINA LLEVA EL REGISTRO DE SU PROPIO TIEMPO.
 - ◆ SE DEBE MANTENER EL TIEMPO DE TODAS LAS MAQUINAS TAN CERCANO COMO SEA POSIBLE.
- SE SUPONE QUE CADA MAQUINA TIENE UN CRONOMETRO QUE PROVOCA UNA INTERRUPCION "H" VECES POR SEGUNDO.
- CUANDO EL CRONOMETRO SE DETIENE, EL MANEJADOR DE INTERRUPCIONES AÑADE "1" A UN RELOJ EN SOFTWARE.
- EL RELOJ EN SOFTWARE MANTIENE UN REGISTRO DEL N° DE MARCAS (INTERRUPCIONES) A PARTIR DE CIERTA FECHA ACORDADA ANTES:
 - ◆ AL VALOR DE ESTE RELOJ SE LO LLAMA "C".

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- CUANDO EL TIEMPO UTC ES “t”, EL VALOR DEL RELOJ EN LA MAQUINA “p” ES “Cp(t)”:
 - ◆ LO IDEAL SERIA QUE “Cp(t)” = “t” PARA TODA “p” Y TODA “t”:
 - “dc/dt” DEBERIA SER “1”.
 - ◆ LO REAL ES QUE LOS CRONOMETROS NO REALIZAN INTERRUPCIONES EXACTAMENTE “H” VECES POR SEGUNDO:
 - POSEEN UN **ERROR RELATIVO** DE APROXIMADAMENTE 10^5 .
 - EL FABRICANTE ESPECIFICA UNA **CONSTANTE ρ** LLAMADA TASA MAXIMA DE ALEJAMIENTO QUE ACOTA EL ERROR.
 - EL CRONOMETRO FUNCIONA DENTRO DE SU **ESPECIFICACION** SI EXISTE UNA CONSTANTE ρ Y SE CUMPLE:
 - $1 - \rho \leq dC / \delta \tau \leq 1 + \rho$.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

31

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- CUANDO EL EMISOR OBTIENE LA RESPUESTA PUEDE HACER QUE SU TIEMPO SEA C_{UTC} .
- UN GRAN PROBLEMA ES QUE EL TIEMPO NO PUEDE CORRER HACIA ATRAS:
 - ◆ C_{UTC} NO PUEDE SER MENOR QUE EL TIEMPO ACTUAL “C” DEL EMISOR.
 - ◆ LA ATENCION DEL REQUERIMIENTO EN EL SERVIDOR DE TIEMPOS REQUIERE UN TIEMPO DEL MANEJADOR DE INTERRUPCIONES.
 - ◆ TAMBIEN SE DEBE CONSIDERAR EL TIEMPO DE TRANSMISION.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

34

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- SI DOS RELOJES SE ALEJAN DE UTC EN LA DIRECCION OPUESTA:
 - ◆ EN UN INSTANTE Δt LUEGO DE LA SINCRONIZACION PODRIAN ESTAR TAN ALEJADOS COMO: $2 \rho \Delta t$.
 - ◆ PARA GARANTIZAR QUE NO DIFIERAN MAS DE δ :
 - SE DEBEN VOLVER A SINCRONIZAR (EN SOFTWARE) AL MENOS CADA $\delta / 2 \rho$ SEGUNDOS.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

32

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- EL CAMBIO DEL RELOJ SE DEBE INTRODUCIR DE MANERA GLOBAL:
 - ◆ SI EL CRONOMETRO GENERA 100 INTERRUPCIONES POR SEGUNDO:
 - CADA INTERRUPCION AÑADE 10 MSEG AL TIEMPO.
 - PARA ATRASAR SOLO AGREGARIA 9 MSEG.
 - PARA ADELANTAR AGREGARIA 11 MSEG.
- LA CORRECCION POR EL TIEMPO DEL SERVIDOR Y EL TIEMPO DE TRANSMISION SE HACE MIDIENDO EN EL EMISOR:
 - ◆ EL TIEMPO INICIAL (ENVIO) T_0 .
 - ◆ EL TIEMPO FINAL (RECEPCION) T_1 .
 - ◆ AMBOS TIEMPOS SE MIDEN CON EL MISMO RELOJ.
- EL TIEMPO DE PROPAGACION DEL MENSAJE SERA $(T_1 - T_0) / 2$.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

35

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- ALGORITMO DE CRISTIAN.
- ES ADECUADO PARA SISTEMAS EN LOS QUE:
 - ◆ UNA MAQUINA TIENE UN **RECEPTOR UTC**:
 - SE LA LLAMA **DESPACHADOR DEL TIEMPO**.
 - ◆ EL OBJETIVO ES SINCRONIZAR TODAS LAS MAQUINAS CON ELLA.
- CADA MAQUINA ENVIA UN MENSAJE AL SERVIDOR PARA SOLICITAR EL TIEMPO ACTUAL:
 - ◆ PERIODICAMENTE, EN UN TIEMPO NO MAYOR QUE $\delta / 2 \rho$ SEGUNDOS.
- EL **DESPACHADOR DEL TIEMPO** RESPONDE PRONTAMENTE CON UN MENSAJE QUE CONTIENE EL TIEMPO ACTUAL C_{UTC} .

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

33

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- SI EL TIEMPO DEL SERVIDOR PARA MANEJAR LA INTERRUPCION Y PROCESAR EL MENSAJE ES “t”:
 - ◆ EL TIEMPO DE PROPAGACION SERA $(T_1 - T_0 - t) / 2$.
- PARA MEJORAR LA PRECISION:
 - ◆ SE TOMAN VARIAS MEDICIONES.
 - ◆ SE DESCARTAN LOS VALORES EXTREMOS.
 - ◆ SE PROMEDIA EL RESTO.
- EL TIEMPO DE PROPAGACION SE SUMA AL TIEMPO DEL SERVIDOR PARA SINCRONIZAR AL EMISOR CUANDO ESTE RECIBE LA RESPUESTA.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

36

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- **ALGORITMO DE BERKELEY.**
- EN EL ALGORITMO DE CRISTIAN EL SERVIDOR DE TIEMPO ES PASIVO.
- EN EL ALGORITMO DE BERKELEY EL SERVIDOR DE TIEMPO:
 - ◆ ES ACTIVO.
 - ◆ REALIZA UN MUESTREO PERIODICO DE TODAS LAS MAQUINAS PARA PREGUNTARLES EL TIEMPO.
 - ◆ CON LAS RESPUESTAS:
 - CALCULA UN TIEMPO PROMEDIO.
 - INDICA A LAS DEMAS MAQUINAS QUE AVANCEN SU RELOJ O DISMINUYAN LA VELOCIDAD DEL MISMO HASTA LOGRAR LA DISMINUCION REQUERIDA.
- ES ADECUADO CUANDO NO SE DISPONE DE UN RECEPTOR UTC.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

37

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- **VARIAS FUENTES EXTERNAS DE TIEMPO.**
- LOS SISTEMAS QUE REQUIEREN UNA SINCRONIZACION MUY PRECISA CON UTC SE PUEDEN EQUIPAR CON VARIOS RECEPTORES DE UTC.
- LAS DISTINTAS FUENTES DE TIEMPO GENERARAN DISTINTOS RANGOS (INTERVALOS DE TIEMPO) DONDE "CAERAN" LOS RESPECTIVOS UTC:
 - ◆ ES NECESARIA UNA SINCRONIZACION.
- COMO LA TRANSMISION NO ES INSTANTANEA SE GENERA UNA CIERTA INCERTIDUMBRE EN EL TIEMPO.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

40

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- **ALGORITMOS CON PROMEDIO.**
- LOS ANTERIORES SON ALGORITMOS CENTRALIZADOS.
- UNA CLASE DE ALGORITMOS DESCENTRALIZADOS DIVIDE EL TIEMPO EN INTERVALOS DE RESINCRONIZACION DE LONGITUD FUA:
 - ◆ EL i -ESIMO INTERVALO:
 - INICIA EN " $T_0 + iR$ " Y VA HASTA " $T_0 + (i + 1)R$ ".
 - " T_0 " ES UN MOMENTO YA ACORDADO EN EL PASADO.
 - " R " ES UN PARAMETRO DEL SISTEMA.
- AL INICIO DE CADA INTERVALO CADA MAQUINA TRANSMITE EL TIEMPO ACTUAL SEGUN SU RELOJ.
- DEBIDO A LA DIFERENTE VELOCIDAD DE LOS RELOJES LAS TRANSMISIONES NO SERAN SIMULTANEAS.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

38

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- CUANDO UN PROCESADOR OBTIENE TODOS LOS RANGOS DE UTC:
 - ◆ VERIFICA SI ALGUNO DE ELLOS ES AJENO A LOS DEMAS Y DE SERLO LO DESCARTA POR SER UN VALOR EXTREMO.
 - ◆ CALCULA LA INTERSECCION (EN EL TIEMPO) DE LOS DEMAS RANGOS.
 - ◆ LA INTERSECCION DETERMINA UN INTERVALO CUYO PUNTO MEDIO SERA EL UTC Y LA HORA DEL RELOJ INTERNO.
- SE DEBEN COMPENSAR LOS RETRASOS DE TRANSMISION Y LAS DIFERENCIAS DE VELOCIDADES DE LOS RELOJES.
- SE DEBE ASEGURAR QUE EL TIEMPO NO CORRA HACIA ATRAS.
- SE DEBE RESINCRONIZAR PERIODICAMENTE DESDE LAS FUENTES EXTERNAS DE UTC.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

41

ALGORITMOS PARA LA SINCRONIZACION DE RELOJES

- **LUEGO DE QUE UNA MAQUINA TRANSMITE SU HORA:**
 - ◆ INICIALIZA UN CRONOMETRO LOCAL PARA REUNIR LAS DEMAS TRANSMISIONES QUE LLEGUEN EN CIERTO INTERVALO " S ".
- CUANDO RECIBE TODAS LAS TRANSMISIONES SE EJECUTA UN ALGORITMO PARA CALCULAR UNA NUEVA HORA PARA LOS RELOJES.
- UNA VARIANTE ES PROMEDIAR LOS VALORES DE TODAS LAS DEMAS MAQUINAS.
- OTRA VARIANTE ES DESCARTAR LOS VALORES EXTREMOS ANTES DE PROMEDIAR (LOS " m " MAYORES Y LOS " m " MENORES).
- UNA MEJORA AL ALGORITMO CONSIDERA LA CORRECCION POR TIEMPOS DE PROPAGACION.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

39

EXCLUSION MUTUA

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

42

EXCLUSION MUTUA

- CUANDO UN PROCESO DEBE **LEER O ACTUALIZAR** CIERTAS ESTRUCTURAS DE DATOS **COMPARTIDAS**:
 - ◆ PRIMERO INGRESA A UNA **REGION CRITICA** PARA:
 - LOGRAR LA **EXCLUSION MUTUA** Y GARANTIZAR QUE NINGUN OTRO PROCESO UTILIZARA LAS ESTRUCTURAS DE DATOS AL MISMO TIEMPO.
- EN SISTEMAS **MONOPROCESADORES** LAS REGIONES CRITICAS SE PROTEGEN CON **SEMAFOROS, MONITORES** Y SIMILARES.
- EN **SISTEMAS DISTRIBUIDOS** LA CUESTION ES MAS **COMPLEJA**.

EXCLUSION MUTUA

- ES UN ESQUEMA SENCILLO, JUSTO Y CON POCOS MENSAJES DE CONTROL.
- LA LIMITANTE ES QUE EL **COORDINADOR PUEDE SER UN CUELLO DE BOTELLA** Y PUEDE FALLAR Y BLOQUEAR A LOS PROCESOS QUE ESPERAN UNA RESPUESTA DE HABILITACION DE ACCESO.

EXCLUSION MUTUA

- UN **ALGORITMO CENTRALIZADO**.
- LA **FORMA MAS DIRECTA** DE LOGRAR LA EXCLUSION MUTUA EN UN SISTEMA DISTRIBUIDO ES **SIMULAR** A LA FORMA EN QUE SE LLEVA A CABO EN UN SISTEMA **MONOPROCESADOR**.
- SE ELIGE UN **PROCESO COORDINADOR**.
- CUANDO UN **PROCESO DESEA** **INGRESAR** A UNA REGION CRITICA:
 - ◆ ENVIA UN **MENSAJE DE SOLICITUD** AL COORDINADOR:
 - INDICANDO LA REGION CRITICA.
 - SOLICITANDO PERMISO DE ACCESO.
 - ◆ **SI NINGUN OTRO PROCESO** ESTA EN ESE MOMENTO EN ESA REGION CRITICA:
 - EL **COORDINADOR** ENVIA UNA RESPUESTA OTORGANDO EL **PERMISO**.
 - ◆ CUANDO LLEGA LA RESPUESTA EL PROCESO SOLICITANTE **ENTRA A LA REGION CRITICA**.

EXCLUSION MUTUA

- UN **ALGORITMO DISTRIBUIDO**.
- EL **OBJETIVO** ES NO TENER UN UNICO PUNTO DE FALLO (EL COORDINADOR CENTRAL).
- UN EJ. ES EL **ALGORITMO DE LAMPORT MEJORADO POR RICART Y AGRAWALA**.
- SE REQUIERE UN **ORDEN TOTAL** DE TODOS LOS EVENTOS EN EL SISTEMA PARA SABER CUAL OCURRIO PRIMERO.
- CUANDO UN **PROCESO DESEA ENTRAR** A UNA REGION CRITICA:
 - ◆ **CONSTRUYE UN MENSAJE** CON EL NOMBRE DE LA REGION CRITICA, SU N° DE PROCESO Y LA HORA ACTUAL.
 - ◆ **ENVIA EL MENSAJE** A TODOS LOS DEMAS PROCESOS Y DE MANERA CONCEPTUAL A EL MISMO.
 - ◆ SE SUPONE QUE **CADA MENSAJE TIENE UN RECONOCIMIENTO**.

EXCLUSION MUTUA

- SI UN PROCESO **PIDE PERMISO** PARA ENTRAR A UNA **REGION CRITICA** YA ASIGNADA A OTRO PROCESO:
 - ◆ EL **COORDINADOR** NO OTORGA EL PERMISO Y ENCOLA EL PEDIDO.
- CUANDO UN **PROCESO SALE** DE LA REGION CRITICA ENVIA UN **MENSAJE AL COORDINADOR** PARA LIBERAR SU ACCESO EXCLUSIVO:
 - ◆ EL **COORDINADOR** EXTRAE EL PRIMER ELEMENTO DE LA **COLA DE SOLICITUDES** DIFERIDAS Y ENVIA A ESE PROCESO UN **MENSAJE OTORGANDO EL PERMISO**:
 - EL PROCESO QUEDA HABILITADO PARA **ACCEDER A LA REGION CRITICA SOLICITADA**.

EXCLUSION MUTUA

- SI EL **RECEPTOR NO ESTA** EN LA REGION CRITICA Y **NO DESEA ENTRAR** A ELLA, ENVIA DE REGRESO UN **MENSAJE OK AL EMISOR**.
- SI EL **RECEPTOR YA ESTA** EN LA REGION CRITICA **NO RESPONDE** Y ENCOLA LA SOLICITUD.
- SI EL RECEPTOR **DESEA ENTRAR** A LA REGION CRITICA PERO AUN **NO LO LOGRO**, COMPARA:
 - ◆ LA MARCA DE **TIEMPO** DEL MENSAJE **RECIBIDO** CON.
 - ◆ LA MARCA CONTENIDA EN EL MENSAJE QUE **ENVIO** A CADA UNO.
 - ◆ LA **MENOR** DE LAS MARCAS **GANA**.
 - ◆ SI EL **MENSAJE RECIBIDO ES MENOR** EL RECEPTOR ENVIA UN **OK**.
 - ◆ SI SU **PROPIO MENSAJE** TIENE UNA MARCA **MENOR** EL RECEPTOR **NO ENVIA NADA** Y ENCOLA EL PEDIDO.

EXCLUSION MUTUA

- LUEGO DE ENVIAR LAS SOLICITUDES UN PROCESO:
 - ◆ ESPERA HASTA QUE ALGUIEN MAS OBTIENE EL PERMISO.
 - ◆ CUANDO LLEGAN TODOS LOS PERMISOS PUEDE ENTRAR A LA REGION CRITICA.
- CUANDO UN PROCESO SALE DE LA REGION CRITICA:
 - ◆ ENVIA MENSAJES OK A TODOS LOS PROCESOS EN SU COLA.
 - ◆ ELIMINA A TODOS LOS ELEMENTOS DE LA COLA.

EXCLUSION MUTUA

- UN IMPORTANTE PROBLEMA ADICIONAL ES QUE:
 - ◆ TODOS LOS PROCESOS PARTICIPAN EN TODAS LAS DECISIONES REFERENTES A LAS ENTRADAS EN LAS REGIONES CRITICAS.
 - ◆ SE SOBRECARGA EL SISTEMA.
- UNA MEJORA CONSISTE EN PERMITIR QUE UN PROCESO ENTRE A UNA REGION CRITICA CON EL PERMISO DE UNA MAYORIA SIMPLE DE LOS DEMAS PROCESOS (EN VEZ DE TODOS):
 - ◆ LUEGO DE QUE UN PROCESO OTORGO EL PERMISO A OTRO PARA ENTRAR A UNA REGION CRITICA:
 - NO PUEDE OTORGAR EL MISMO PERMISO A OTRO PROCESO HASTA QUE EL PRIMERO LIBERE SU PERMISO.

EXCLUSION MUTUA

- LA EXCLUSION MUTUA QUEDA GARANTIZADA SIN BLOQUEO NI INANICION.
- EL N° DE MENSAJES NECESARIOS POR ENTRADA ES " $2(n - 1)$ ", SIENDO "n" EL N° TOTAL DE PROCESOS EN EL SISTEMA.
- NO EXISTE UN UNICO PUNTO DE FALLO SINO "n":
 - ◆ SI CUALQUIER PROCESO FALLA NO RESPONDERA A LAS SOLICITUDES.
 - ◆ LA FALTA DE RESPUESTA SE INTERPRETARA COMO NEGACION DE ACCESO:
 - SE BLOQUEARAN LOS SIGUIENTES INTENTOS DE LOS DEMAS PROCESOS POR ENTRAR A TODAS LAS REGIONES CRITICAS.

EXCLUSION MUTUA

- UN ALGORITMO DE ANILLO DE FICHAS (TOKEN RING).
- LOS PROCESOS SE ORGANIZAN POR SOFTWARE FORMANDO UN ANILLO LOGICO ASIGNANDOSE A CADA PROCESO UNA POSICION EN EL ANILLO.
- CADA PROCESO SABE CUAL ES EL SIGUIENTE LUEGO DE EL.
- AL INICIALIZAR EL ANILLO SE LE DA AL PROCESO "0" UNA FICHA (TOKEN) QUE CIRCULA EN TODO EL ANILLO:
 - ◆ SE TRANSFIERE DEL PROCESO "k" AL "k + 1" EN MENSAJES PUNTUALES.

EXCLUSION MUTUA

- SE INCREMENTA LA PROBABILIDAD DE FALLO EN "n" VECES Y TAMBIEN EL TRAFICO EN LA RED.
- SE PUEDE SOLUCIONAR EL BLOQUEO SI:
 - ◆ EL EMISOR ESPERA Y SIGUE INTENTANDO HASTA QUE REGRESA UNA RESPUESTA O.
 - ◆ EL EMISOR CONCLUYE QUE EL DESTINATARIO ESTA FUERA DE SERVICIO.
- OTRO PROBLEMA ES QUE:
 - ◆ SE UTILIZARA UNA PRIMITIVA DE COMUNICACION EN GRUPO O.
 - ◆ CADA PROCESO DEBE MANTENER LA LISTA DE MIEMBROS DEL GRUPO, INCLUYENDO LOS PROCESOS QUE INGRESAN, LOS QUE SALEN Y LOS QUE FALLAN.
 - ◆ SE COMPLICA PARA GRAN N° DE PROCESOS.

EXCLUSION MUTUA

- CUANDO UN PROCESO OBTIENE LA FICHA DE SU VECINO VERIFICA SI INTENTA ENTRAR A UNA REGION CRITICA:
 - ◆ EN CASO POSITIVO:
 - EL PROCESO ENTRA A LA REGION CRITICA, HACE EL PROCESO NECESARIO Y SALE DE ELLA.
 - DESPUES DE SALIR PASA LA FICHA A LO LARGO DEL ANILLO:
 - NO SE PUEDE ENTRAR A UNA SEGUNDA REGION CRITICA CON LA MISMA FICHA (TOKEN O PERMISO).
 - ◆ EN CASO NEGATIVO:
 - LA VUELVE A PASAR.

EXCLUSION MUTUA

- EN UN INSTANTE DADO **SOLO UN PROCESO** PUEDE ESTAR EN UNA **REGION CRITICA**.
- SI LA **FICHA SE PIERDE** DEBE SER **REGENERADA**, PERO ES DIFICIL **DETECTAR SU PERDIDA**:
 - ◆ LA CANTIDAD DE **TIEMPO ENTRE LAS APARICIONES** SUCESIVAS DE LA FICHA EN LA RED **NO ESTA ACOTADA**:
 - ES DIFICIL DECIDIR SI ESTA **PERDIDA O DEMORADA** EN ALGUN PROCESO QUE NO LA LIBERA.
- LA **FALLA DE UN PROCESO** ES DETECTADA CUANDO SU VECINO **INTENTA SIN EXITO PASARLE LA FICHA**:
 - ◆ SE LO DEBE **ELIMINAR DEL GRUPO** Y PASAR LA FICHA AL SIGUIENTE PROCESO ACTIVO:
 - TODOS LOS PROCESOS DEBEN **MANTENER LA CONFIGURACION ACTUAL DEL ANILLO**.

ALGORITMOS DE ELECCION

- EL ALGORITMO DEL GRANDULON O DE GARCIA-MOLINA.
- UN PROCESO “P” **INICIA UNA ELECCION** CUANDO OBSERVA QUE EL **COORDINADOR YA NO RESPONDE** A LAS SOLICITUDES.
- “P” REALIZA UNA ELECCION DE LA **SIGUIENTE MANERA**:
 - ◆ ENVIA UN MENSAJE **ELECCION** A LOS DEMAS PROCESOS CON UN N° MAYOR.
 - ◆ SI **NADIE RESPONDE** ASUME QUE GANA LA ELECCION Y SE CONVIERTE EN EL **NUEVO COORDINADOR**.
 - ◆ SI UN PROCESO CON UN N° **MAYOR RESPONDE**, TOMA EL **CONTROL** Y EL TRABAJO DE “P” **TERMINA**.
- UN PROCESO PUEDE **RECIBIR EN CUALQUIER MOMENTO UN MENSAJE ELECCION** DE OTROS PROCESOS CON UN N° **MENOR**:
 - ◆ ENVIA DE REGRESO UN MENSAJE **OK** AL EMISOR PARA INDICAR QUE ESTA VIVO Y QUE **TOMARA EL CONTROL**.
 - ◆ **REALIZA UNA ELECCION** SALVO QUE YA ESTE HACIENDO ALGUNA.

ALGORITMOS DE ELECCION

ALGORITMOS DE ELECCION

- EN **CIERTO MOMENTO** TODOS LOS PROCESOS HAN DECLINADO ANTE UNO DE ELLOS, QUE SERA EL **NUEVO COORDINADOR**:
 - ◆ ENVIA UN MENSAJE **COORDINADOR** A TODOS LOS PROCESOS PARA ANUNCIARLO.
- SI UN **PROCESO INACTIVO** SE ACTIVA REALIZA UNA **ELECCION**:
 - ◆ SI EL TIENE EL N° **MAS ALTO** SERA EL **NUEVO COORDINADOR**:
 - **SIEMPRE GANA EL PROCESO QUE POSEE EL N° MAYOR**, DE AHI EL NOMBRE “**ALGORITMO DEL GRANDULON**”.

ALGORITMOS DE ELECCION

- SON LOS ALGORITMOS PARA LA **ELECCION DE UN PROCESO COORDINADOR**, INICIADOR, SECUENCIADOR, ETC.
- EL **OBJETIVO** DE UN ALGORITMO DE ELECCION ES GARANTIZAR QUE **INICIADA UNA ELECCION ESTA CONCLUYA** CON EL ACUERDO DE TODOS LOS PROCESOS CON RESPECTO A LA **IDENTIDAD DEL NUEVO COORDINADOR**.

ALGORITMOS DE ELECCION

- UN ALGORITMO DE ANILLO.
- SE SUPONE QUE LOS **PROCESOS TIENEN UN ORDEN FISICO O LOGICO**:
 - ◆ CADA PROCESO **CONOCE A SU SUCESOR**.
- CUANDO ALGUN **PROCESO OBSERVA** QUE EL **COORDINADOR NO FUNCIONA**:
 - ◆ CONSTRUYE UN MENSAJE **ELECCION** CON SU PROPIO N° DE PROCESO.
 - ◆ ENVIA EL MENSAJE A SU SUCESOR.
 - ◆ SI EL SUCESOR **ESTA INACTIVO**:
 - EL EMISOR VA HACIA EL **SIGUIENTE N° DEL ANILLO** O AL SIGUIENTE DE ESTE.
 - CONTINUA **HASTA LOCALIZAR UN PROCESO EN EJECUCION**.
 - EN CADA PASO, EL EMISOR **AÑADE SU PROPIO N° DE PROCESO** A LA LISTA EN EL MENSAJE.

ALGORITMOS DE ELECCION

- ◆ EN CIERTO MOMENTO EL MENSAJE REGRESA AL PROCESO QUE LO INICIO:
 - EL PROCESO LO RECONOCE AL RECIBIR UN MENSAJE CON SU PROPIO N° DE PROCESO.
- ◆ EL MENSAJE DE *ELECCION* SE TRANSFORMA EN MENSAJE *COORDINADOR* Y CIRCULA NUEVAMENTE:
 - INFORMA A LOS DEMAS PROCESOS:
 - QUIEN ES EL **COORDINADOR**:
 - EL MIEMBRO DE LA LISTA CON EL N° MAYOR.
 - QUIENES SON LOS MIEMBROS DEL NUEVO ANILLO.
- ◆ CONCLUIDA LA RONDA DE INFORMACION EL MENSAJE *COORDINADOR* SE ELIMINA Y CONTINUAN LOS PROCESOS.

TRANSACCIONES ATOMICAS

- LA PRINCIPAL PROPIEDAD DE LA TRANSACCION ATOMICA ES EL "TODO O NADA":
 - ◆ O SE HACE TODO LO QUE SE TENIA QUE HACER COMO UNA UNIDAD O NO SE HACE NADA.
- ◆ EJEMPLO:
 - UN CLIENTE LLAMA AL BANCO MEDIANTE UNA PC CON UN MODEM PARA:
 - RETIRAR DINERO DE UNA CUENTA.
 - DEPOSITAR EL DINERO EN OTRA CUENTA.
 - LA OPERACION TIENE DOS ETAPAS.
 - SI LA CONEXION TELEFONICA FALLA LUEGO DE LA PRIMER ETAPA PERO ANTE DE LA SEGUNDA:
 - HABRA UN RETIRO PERO NO UN DEPOSITO.

TRANSACCIONES ATOMICAS

TRANSACCIONES ATOMICAS

- LA SOLUCION CONSISTE EN AGRUPAR LAS DOS OPERACIONES EN UNA TRANSACCION ATOMICA:
 - LAS DOS OPERACIONES TERMINARIAN O NO TERMINARIA NINGUNA.
 - SE DEBE REGRESAR AL ESTADO INICIAL SI LA TRANSACCION NO PUEDE CONCLUIR.

TRANSACCIONES ATOMICAS

- LAS TECNICAS DE SINCRONIZACION YA VISTAS SON DE BAJO NIVEL:
 - ◆ EL PROGRAMADOR DEBE ENFRENTARSE DIRECTAMENTE CON LOS DETALLES DE:
 - LA EXCLUSION MUTUA.
 - EL MANEJO DE LAS REGIONES CRITICAS.
 - LA PREVENCION DE BLOQUEOS.
 - LA RECUPERACION DE FALLAS.
- SE PRECISAN TECNICAS DE ABSTRACCION DE MAYOR NIVEL QUE:
 - ◆ OCULTEN ESTOS ASPECTOS TECNICOS.
 - ◆ PERMITAN A LOS PROGRAMADORES CONCENTRARSE EN LOS ALGORITMOS Y LA FORMA EN QUE LOS PROCESOS TRABAJAN JUNTOS EN PARALELO.
- TAL ABSTRACCION LA LLAMAREMOS TRANSACCION ATOMICA, TRANSACCION O ACCION ATOMICA.

EL MODELO DE TRANSACCION

EL MODELO DE TRANSACCION

- SUPONDREMOS QUE:
 - ◆ EL SISTEMA CONSTA DE VARIOS PROCESOS INDEPENDIENTES QUE PUEDEN FALLAR ALEATORIAMENTE.
 - ◆ EL SOFTWARE SUBYACENTE MANEJA TRANSPARENTEMENTE LOS ERRORES DE COMUNICACION.

EL MODELO DE TRANSACCION

- PRIMITIVAS DE TRANSACCION.
- DEBEN SER PROPORCIONADAS POR EL SISTEMA OPERATIVO O POR EL SISTEMA DE TIEMPO DE EJECUCION DEL LENGUAJE.
- EJEMPLOS:
 - ◆ *BEGIN_TRANSACTION*: LOS COMANDOS SIGUIENTES FORMAN UNA TRANSACCION.
 - ◆ *END_TRANSACTION*: TERMINA LA TRANSACCION Y SE INTENTA UN COMPROMISO.
 - ◆ *ABORT_TRANSACTION*: SE ELIMINA LA TRANSACCION; SE RECUPERAN LOS VALORES ANTERIORES.
 - ◆ *READ*: SE LEEN DATOS DE UN ARCHIVO (O ALGUN OTRO OBJETO).
 - ◆ *WRITE*: SE ESCRIBEN DATOS EN UN ARCHIVO (O ALGUN OTRO OBJETO).

EL MODELO DE TRANSACCION

- ALMACENAMIENTO ESTABLE.
- SE PUEDE IMPLANTAR CON UNA PAREJA DE DISCOS COMUNES.
- CADA BLOQUE DE LA UNIDAD 2 ES UNA COPIA EXACTA (ESPEJO) DEL BLOQUE CORRESPONDIENTE EN LA UNIDAD 1.
- CUANDO SE ACTUALIZA UN BLOQUE:
 - ◆ PRIMERO SE ACTUALIZA Y VERIFICA EL BLOQUE DE LA UNIDAD 1.
 - ◆ LUEGO SE ACTUALIZA Y VERIFICA EL BLOQUE DE LA UNIDAD 2.
- SI EL SISTEMA FALLA LUEGO DE ACTUALIZAR LA UNIDAD 1 Y ANTES DE ACTUALIZAR LA UNIDAD 2:
 - ◆ LUEGO DE LA RECUPERACION SE PUEDEN COMPARAR AMBOS DISCOS BLOQUE POR BLOQUE:
 - SE PUEDE ACTUALIZAR LA UNIDAD 2 EN FUNCION DE LA 1.

EL MODELO DE TRANSACCION

- LAS OPERACIONES ENTRE *BEGIN* Y *END* FORMAN EL CUERPO DE LA TRANSACCION Y DEBEN EJECUTARSE TODAS O NINGUNA DE ELLAS:
 - ◆ PUEDEN SER LLAMADAS AL SISTEMA, PROCEDIMIENTO DE BIBLIOTECA O ENUNCIADOS EN UN LENGUAJE.

EL MODELO DE TRANSACCION

- SI SE DETECTA EL DETERIORO ESPONTANEO DE UN BLOQUE:
 - ◆ SE LO REGENERA PARTIENDO DEL BLOQUE CORRESPONDIENTE EN LA OTRA UNIDAD.
- UN ESQUEMA DE ESTE TIPO ES ADECUADO PARA LAS APLICACIONES QUE REQUIEREN DE UN ALTO GRADO DE TOLERANCIA DE FALLOS:
 - ◆ EJ.: LAS TRANSACCIONES ATOMICAS.

EL MODELO DE TRANSACCION

- PROPIEDADES DE LAS TRANSACCIONES.
- LAS PROPIEDADES FUNDAMENTALES SON:
 - ◆ *SERIALIZACION*:
 - LAS TRANSACCIONES CONCURRENTES NO INTERFIEREN ENTRE SI.
 - ◆ *ATOMICIDAD*:
 - PARA EL MUNDO EXTERIOR, LA TRANSACCION OCURRE DE MANERA INDIVISIBLE.
 - ◆ *PERMANENCIA*:
 - UNA VEZ COMPROMETIDA UNA TRANSACCION, LOS CAMBIOS SON PERMANENTES.

EL MODELO DE TRANSACCION

- LA *SERIALIZACION* GARANTIZA QUE SI DOS O MAS TRANSACCIONES SE EJECUTAN AL MISMO TIEMPO:
 - ◆ EL RESULTADO FINAL APARECE COMO SI TODAS LAS TRANSACCIONES SE EJECUTASEN DE MANERA SECUENCIAL EN CIERTO ORDEN:
 - PARA C / U DE ELLAS Y PARA LOS DEMAS PROCESOS.
- LA *ATOMICIDAD* GARANTIZA QUE CADA TRANSACCION NO OCURRE O BIEN SE REALIZA EN SU TOTALIDAD:
 - ◆ SE PRESENTA COMO UNA ACCION INDIVISIBLE E INSTANTANEA.
- LA *PERMANENCIA* SE REFIERE A QUE UNA VEZ COMPROMETIDA UNA TRANSACCION:
 - ◆ SIGUE ADELANTE Y LOS RESULTADOS SON PERMANENTES.

IMPLANTACION DEL MODELO DE TRANSACCION

- EXISTEN VARIOS METODOS DE IMPLANTACION.
- **ESPACIO DE TRABAJO PARTICULAR.**
- CONSISTE EN QUE CUANDO UN **PROCESO INICIA** UNA TRANSACCION SE LE OTORGA UN **ESPACIO DE TRABAJO PARTICULAR**:
 - ◆ CONTIENE TODOS LOS **ARCHIVOS** (Y OTROS OBJETOS) A LOS CUALES TIENE ACCESO.
 - ◆ LAS **LECTURAS Y ESCRITURAS** IRAN A ESTE ESPACIO HASTA QUE LA TRANSACCION SE **COMPLETE O ABORTE**:
 - EL “**ESPACIO REAL**” ES EL SISTEMA DE ARCHIVOS NORMAL.
 - ◆ SIGNIFICA **ALTO CONSUMO DE RECURSOS** POR LAS COPIAS DE LOS OBJETOS AL ESPACIO DE TRABAJO PARTICULAR.

EL MODELO DE TRANSACCION

- **TRANSACCIONES ANIDADAS.**
- SE PRESENTAN CUANDO LAS TRANSACCIONES PUEDEN CONTENER SUBTRANSACCIONES (PROCESOS HIJOS) QUE:
 - ◆ SE EJECUTEN EN **PARALELO** ENTRE SI EN PROCESADORES DISTINTOS.
 - ◆ PUEDEN ORIGINAR NUEVAS **SUBTRANSACCIONES**.

IMPLANTACION DEL MODELO DE TRANSACCION

- CUANDO UN **PROCESO INICIA** UNA TRANSACCION:
 - ◆ BASTA **CREAR UN ESPACIO DE TRABAJO PARTICULAR** PARA EL QUE SEA **VACIO** EXCEPTO POR UN **APUNTADEOR** DE REGRESO AL **ESPACIO DE TRABAJO DE SU PROCESO PADRE**.
- PARA UNA **TRANSACCION DEL NIVEL SUPERIOR** EL ESPACIO DE TRABAJO DEL **PADRE** ES EL SISTEMA DE ARCHIVOS “**REAL**”.
- CUANDO EL PROCESO **ABRE UN ARCHIVO** PARA **LECTURA**:
 - ◆ SE SIGUEN LOS **APUNTADEORES DE REGRESO** HASTA LOCALIZAR EL ARCHIVO EN EL ESPACIO DE TRABAJO DEL **PADRE** (O ALGUN ANTECESOR).

IMPLANTACION DEL MODELO DE TRANSACCION

IMPLANTACION DEL MODELO DE TRANSACCION

- CUANDO SE **ABRE UN ARCHIVO** PARA **ESCRITURA**:
 - ◆ SE LO **LOCALIZA** DE MANERA SIMILAR QUE PARA LECTURA.
 - ◆ SE **COPIA** EN PRIMER LUGAR AL ESPACIO DE TRABAJO PARTICULAR:
 - UNA **OPTIMIZACION** CONSISTE EN COPIAR SOLO EL **INDICE DEL ARCHIVO** EN EL ESPACIO DE TRABAJO PARTICULAR:
 - EL INDICE ES EL BLOQUE DE DATOS ASOCIADO A CADA ARCHIVO E INDICA LA **LOCALIZACION** DE SUS **BLOQUES** EN EL DISCO.
 - ES EL **NODO-I** CORRESPONDIENTE.

IMPLANTACION DEL MODELO DE TRANSACCION

- LA LECTURA POR MEDIO DEL INDICE PARTICULAR (DEL ESPACIO DE TRABAJO PARTICULAR) NO ES PROBLEMÁTICA:
 - ◆ LAS DIRECCIONES EN DISCO A LAS QUE REFERENCIA SON LAS ORIGINALES.
- LA MODIFICACION DE UN BLOQUE DE UN ARCHIVO REQUIERE:
 - ◆ HACER UNA COPIA DEL BLOQUE.
 - ◆ INSERTAR EN EL INDICE LA DIRECCION DE LA COPIA.
- LA MODIFICACION SOBRE LA COPIA NO AFECTA AL BLOQUE ORIGINAL.
- UN TRATAMIENTO SIMILAR SE DA AL AGREGADO DE BLOQUES:
 - ◆ LOS NUEVOS BLOQUES SE LLAMAN BLOQUES SOMBRA (SHADOW BLOCKS).

IMPLANTACION DEL MODELO DE TRANSACCION

- SI LA TRANSACCION TIENE ÉXITO Y SE HACE UN COMPROMISO:
 - ◆ SE ESCRIBE UN REGISTRO DEL COMPROMISO EN LA BITÁCORA.
 - ◆ LAS ESTRUCTURAS DE DATOS NO TIENEN QUE MODIFICARSE:
 - YA HAN SIDO ACTUALIZADAS.
- SI LA TRANSACCION ABORTA:
 - ◆ SE PUEDE UTILIZAR LA BITÁCORA PARA RESPALDO DEL ESTADO ORIGINAL:
 - A PARTIR DEL FINAL Y HACIA ATRÁS:
 - SE LEE CADA REGISTRO DE LA BITÁCORA.
 - SE DESHACE CADA CAMBIO DESCRIPTO EN EL.
 - ◆ ESTA ACCIÓN SE DENOMINA RETROALIMENTACION.
 - ◆ POR MEDIO DE LA BITÁCORA SE PUEDE:
 - ◆ IR HACIA ADELANTE (REALIZAR LA TRANSACCION).
 - ◆ IR HACIA ATRÁS (DESHACER LA TRANSACCION).

IMPLANTACION DEL MODELO DE TRANSACCION

- EL PROCESO QUE EJECUTA LA TRANSACCION VE EL ARCHIVO MODIFICADO PERO LOS DEMÁS PROCESOS VEN EL ARCHIVO ORIGINAL.
- SI LA TRANSACCION ABORTA (TERMINA ANORMALMENTE):
 - ◆ EL ESPACIO DE TRABAJO PARTICULAR SE ELIMINA.
 - ◆ LOS BLOQUES PARTICULARES A LOS QUE APUNTA SE COLOCAN NUEVAMENTE EN LA LISTA DE BLOQUES LIBRES.
- SI LA TRANSACCION SE COMPROMETE (TERMINA NORMALMENTE):
 - ◆ LOS INDICES PARTICULARES SE DESPLAZAN AL ESPACIO DE TRABAJO DEL PADRE DE MANERA ATÓMICA.
 - ◆ LOS BLOQUES QUE NO SON ALCANZABLES SE COLOCAN EN LA LISTA DE BLOQUES LIBRES.

IMPLANTACION DEL MODELO DE TRANSACCION

- PROTOCOLO DE COMPROMISO DE DOS FASES (TWO - PHASE COMMIT).
- UNO DE LOS PROCESOS QUE INTERVIENEN FUNCIONA COMO EL COORDINADOR.
- EL COORDINADOR ESCRIBE UNA ENTRADA EN LA BITÁCORA PARA INDICAR QUE INICIA EL PROTOCOLO.
- EL COORDINADOR ENVÍA A C / U DE LOS PROCESOS RELACIONADOS (SUBORDINADOS) UN MENSAJE PARA QUE ESTÉN PREPARADOS PARA EL COMPROMISO.
- CUANDO UN SUBORDINADO RECIBE EL MENSAJE:
 - ◆ VERIFICA SI ESTÁ LISTO PARA COMPROMETERSE.
 - ◆ ESCRIBE UNA ENTRADA EN LA BITÁCORA.
 - ◆ ENVÍA DE REGRESO SU DECISIÓN.

IMPLANTACION DEL MODELO DE TRANSACCION

- BITÁCORA DE ESCRITURA ANTICIPADA.
- ESTE MÉTODO TAMBIÉN SE DENOMINA LISTA DE INTENCIONES.
- LOS ARCHIVOS REALMENTE SE MODIFICAN PERO ANTES DE CAMBIAR CUALQUIER BLOQUE:
 - ◆ SE GRABA UN REGISTRO EN LA BITÁCORA ("LOG") DE ESCRITURA ANTICIPADA EN UN ESPACIO DE ALMACENAMIENTO ESTABLE:
 - SE INDICA LA TRANSACCION QUE PRODUCE EL CAMBIO, EL ARCHIVO Y BLOQUE MODIFICADOS Y LOS VALORES ANTERIOR Y NUEVO.

IMPLANTACION DEL MODELO DE TRANSACCION

- CUANDO EL COORDINADOR HA RECIBIDO TODAS LAS RESPUESTAS SABE SI DEBE ESTABLECER EL COMPROMISO O ABORTAR:
 - ◆ SI TODOS LOS PROCESOS ESTÁN LISTOS PARA COMPROMETERSE CIERRA LA TRANSACCION.
 - ◆ SI ALGUNO DE LOS PROCESOS NO SE COMPROMETE (O NO RESPONDE) LA TRANSACCION SE ABORTA.
 - ◆ EL COORDINADOR:
 - ESCRIBE UNA ENTRADA EN LA BITÁCORA.
 - ENVÍA UN MENSAJE A CADA SUBORDINADO PARA INFORMARLE DE LA DECISIÓN.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- EL SISTEMA DE TRANSACCIONES GENERALMENTE ADQUIERE Y LIBERA LAS CERRADURAS SIN ACCION POR PARTE DEL PROGRAMADOR.
- UNA MEJORA CONSISTE EN DISTINGUIR LAS CERRADURAS PARA LECTURA DE LAS CERRADURAS PARA ESCRITURA.
- UNA CERRADURA PARA LECTURA NO IMPIDE OTRAS CERRADURAS PARA LECTURA:
 - ◆ LAS CERRADURAS PARA LECTURA SE COMPARTEN.
- UNA CERRADURA PARA ESCRITURA SI IMPIDE OTRAS CERRADURAS (DE LECTURA O DE ESCRITURA):
 - ◆ LAS CERRADURAS PARA ESCRITURA NO SE COMPARTEN:
 - DEBEN SER EXCLUSIVAS.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- LOS ALGORITMOS DE CONTROL DE CONCURRENCIA SON NECESARIOS CUANDO SE EJECUTAN VARIAS TRANSACCIONES DE MANERA SIMULTANEA:
 - ◆ EN DISTINTOS PROCESOS.
 - ◆ EN DISTINTOS PROCESADORES.
- LOS PRINCIPALES ALGORITMOS SON:
 - ◆ EL DE LA CERRADURA.
 - ◆ EL DEL CONTROL OPTIMISTA DE LA CONCURRENCIA.
 - ◆ EL DE LAS MARCAS DE TIEMPO.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- EL ELEMENTO POR CERRAR PUEDE SER UN ARCHIVO, UN REGISTRO, UN CAMPO, ETC.:
 - ◆ LO RELATIVO AL TAMAÑO DEL ELEMENTO POR CERRAR SE LLAMA LA GRANULARIDAD DE LA CERRADURA.
- MIENTRAS MAS FINA SEA LA GRANULARIDAD:
 - ◆ PUEDE SER MAS PRECISA LA CERRADURA.
 - ◆ SE PUEDE LOGRAR UN MAYOR PARALELISMO EN EL ACCESO AL RECURSO.
 - ◆ SE REQUIERE UN MAYOR N° DE CERRADURAS.
- GENERALMENTE SE UTILIZA LA CERRADURA DE DOS FASES:
 - ◆ EL PROCESO ADQUIERE TODAS LAS CERRADURAS NECESARIAS DURANTE LA FASE DE CRECIMIENTO.
 - ◆ EL PROCESO LAS LIBERA EN LA FASE DE REDUCCION.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- CERRADURA (LOCKING).
- CUANDO UN PROCESO DEBE LEER O ESCRIBIR EN UN ARCHIVO (U OTRO OBJETO) COMO PARTE DE UNA TRANSACCION:
 - ◆ PRIMERO CIERRA EL ARCHIVO.
- LA CERRADURA SE PUEDE HACER MEDIANTE:
 - ◆ UN UNICO MANEJADOR CENTRALIZADO DE CERRADURAS.
 - ◆ UN MANEJADOR LOCAL DE CERRADURAS EN CADA MAQUINA.
- EL MANEJADOR DE CERRADURAS:
 - ◆ MANTIENE UNA LISTA DE LOS ARCHIVOS CERRADOS.
 - ◆ RECHAZA TODOS LOS INTENTOS POR CERRAR ARCHIVOS YA CERRADOS POR OTROS PROCESOS.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- SE DEBEN EVITAR SITUACIONES DE ABORTO EN CASCADA:
 - ◆ SE GRABA EN UN ARCHIVO Y LUEGO SE LIBERA SU CERRADURA.
 - ◆ OTRA TRANSACCION LO CIERRA, REALIZA SU TRABAJO Y LUEGO ESTABLECE UN COMPROMISO.
 - ◆ LA TRANSACCION ORIGINAL ABORTA.
 - ◆ LA SEGUNDA TRANSACCION (YA COMPROMETIDA) DEBE DESHACERSE:
 - SUS RESULTADOS SE BASAN EN UN ARCHIVO QUE NO DEBERIA HABER VISTO CUANDO LO HIZO.
- LAS CERRADURAS COMUNES Y DE DOS FASES PUEDEN PROVOCAR BLOQUEOS CUANDO DOS PROCESOS INTENTAN ADQUIRIR LA MISMA PAREJA DE CERRADURAS PERO EN ORDEN OPUESTO:
 - ◆ SE DEBEN UTILIZAR TECNICAS DE PREVENCIÓN Y DE DETECCIÓN DE BLOQUEOS PARA SUPERAR EL PROBLEMA.

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- **CONTROL OPTIMISTA DE LA CONCURRENCIA.**
- LA IDEA ES MUY SENCILLA:
 - ◆ SE SIGUE ADELANTE Y SE HACE TODO LO QUE SE DEBA HACER, SIN PRESTAR ATENCION A LO QUE HACEN LOS DEMAS.
 - ◆ SE ACTUA A POSTERIORI SI SE PRESENTA ALGUN PROBLEMA.
- SE MANTIENE UN REGISTRO DE LOS ARCHIVOS LEIDOS O GRABADOS.
- EN EL MOMENTO DEL COMPROMISO:
 - ◆ SE VERIFICAN TODAS LAS DEMAS TRANSACCIONES PARA VER SI ALGUNO DE LOS ARCHIVOS HA SIDO MODIFICADO DESDE EL INICIO DE LA TRANSACCION:
 - SI ESTO OCURRE LA TRANSACCION ABORTA.
 - SI ESTO NO OCURRE SE REALIZA EL COMPROMISO.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

91

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- SI LA MARCA DE TIEMPO DE LA TRANSACCION ACTIVA ES MENOR QUE LA DEL ARCHIVO QUE INTENTA ACCEDER:
 - ◆ UNA TRANSACCION INICIADA POSTERIORMENTE HA ACCEDIDO AL ARCHIVO Y HA EFECTUADO UN COMPROMISO.
 - ◆ LA TRANSACCION ACTIVA SE HA REALIZADO TARDE Y SE ABORTA.
- EN EL METODO DE LAS MARCAS NO PREOCUPA QUE LAS TRANSACCIONES CONCURRENTES UTILICEN LOS MISMOS ARCHIVOS:
 - ◆ SI IMPORTA QUE LA TRANSACCION CON EL N° MENOR ESTE EN PRIMER LUGAR.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

94

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- LAS PRINCIPALES VENTAJAS SON:
 - ◆ AUSENCIA DE BLOQUEOS.
 - ◆ PARALELISMO MAXIMO YA QUE NO SE ESPERAN CERRADURAS.
- LA PRINCIPAL DESVENTAJA ES:
 - ◆ RE-EJECUCION DE LA TRANSACCION EN CASO DE FALLA.
 - ◆ LA PROBABILIDAD DE FALLO PUEDE CRECER SI LA CARGA DE TRABAJO ES MUY ALTA.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

92

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- LAS MARCAS DE TIEMPO TIENEN PROPIEDADES DISTINTAS A LAS DE LOS BLOQUEOS:
 - ◆ UNA TRANSACCION ABORTA CUANDO ENCUENTRA UNA MARCA MAYOR (POSTERIOR).
 - ◆ EN IGUALES CIRCUNSTANCIAS Y EN UN ESQUEMA DE CERRADURAS PODRIA ESPERAR O CONTINUAR INMEDIATAMENTE.
- LAS MARCAS DE TIEMPO SON LIBRES DE BLOQUEOS:
 - ◆ ES UNA GRAN VENTAJA.
- RESUMEN.
- LOS DIFERENTES ESQUEMAS OFRECEN DISTINTAS VENTAJAS PERO EL PROBLEMA PRINCIPAL ES LA GRAN COMPLEJIDAD DE SU IMPLANTACION.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

95

CONTROL DE CONCURRENCIA EN EL MODELO DE TRANSACCION

- **MARCAS DE TIEMPO.**
- SE ASOCIA A CADA TRANSACCION UNA MARCA DE TIEMPO AL INICIAR (*BEGIN_TRANSACTION*).
- SE GARANTIZA QUE LAS MARCAS SON UNICAS MEDIANTE EL ALGORITMO DE LAMPORT.
- CADA ARCHIVO DEL SISTEMA TIENE ASOCIADAS UNA MARCA DE TIEMPO PARA LA LECTURA Y OTRA PARA LA ESCRITURA:
 - ◆ INDICAN LA ULTIMA TRANSACCION COMPROMETIDA QUE REALIZO LA LECTURA O ESCRITURA.
- CUANDO UN PROCESO INTENTE ACCEDER A UN ARCHIVO:
 - ◆ LO LOGRARA SI LAS MARCAS DE TIEMPO DE LECTURA Y ESCRITURA SON MENORES (MAS ANTIGUAS) QUE LA MARCA DE LA TRANSACCION ACTIVA.

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

93

BLOQUEOS EN SISTEMAS DISTRIBUIDOS

SINCRONIZACION EN SISTEMAS DISTRIBUIDOS

96

BLOQUEOS EN SISTEMAS DISTRIBUIDOS

- SON PEORES QUE LOS BLOQUEOS EN SISTEMAS MONOPROCESADOR:
 - ◆ SON MAS DIFICILES DE EVITAR, PREVENIR, DETECTAR Y SOLUCIONAR.
 - ◆ TODA LA INFORMACION RELEVANTE ESTA DISPERSA EN MUCHAS MAQUINAS.
- SON ESPECIALMENTE CRITICOS EN SISTEMAS DE BASES DE DATOS DISTRIBUIDOS.

DETECCION DISTRIBUIDA DE BLOQUEOS

BLOQUEOS EN SISTEMAS DISTRIBUIDOS

- LAS ESTRATEGIAS USUALES PARA EL MANEJO DE LOS BLOQUEOS SON:
 - ◆ ALGORITMO DEL AVESTRUZ:
 - ⇒ IGNORAR EL PROBLEMAS.
 - ◆ DETECCION:
 - ⇒ PERMITIR QUE OCURRAN LOS BLOQUEOS, DETECTARLOS E INTENTAR RECUPERARSE DE ELLOS.
 - ◆ PREVENCIÓN:
 - ⇒ HACER QUE LOS BLOQUEOS SEAN IMPOSIBLES DESDE EL PUNTO DE VISTA ESTRUCTURAL.
 - ◆ EVITARLOS:
 - ⇒ EVITAR LOS BLOQUEOS MEDIANTE LA ASIGNACION CUIDADOSA DE LOS RECURSOS.

DETECCION DISTRIBUIDA DE BLOQUEOS

- CUANDO SE DETECTA UN BLOQUEO EN UN S. O. CONVENCIONAL SE RESUELVE ELIMINANDO UNO O MAS PROCESOS.
- CUANDO SE DETECTA UN BLOQUEO EN UN SISTEMA BASADO EN TRANSACCIONES ATOMICAS SE RESUELVE ABORTANDO UNA O MAS TRANSACCIONES:
 - ◆ EL SISTEMA RESTAURA EL ESTADO QUE TENIA ANTES DE INICIAR LA TRANSACCION.
 - ◆ LA TRANSACCION PUEDE VOLVER A COMENZAR.
- LAS CONSECUENCIAS DE LA ELIMINACION DE UN PROCESO:
 - ◆ SON MUCHO MENOS SEVERAS SI SE UTILIZAN LAS TRANSACCIONES QUE EN CASO DE QUE NO SE UTILICEN.

BLOQUEOS EN SISTEMAS DISTRIBUIDOS

- EL ALGORITMO DEL AVESTRUZ MERECE LAS MISMAS CONSIDERACIONES QUE EN EL CASO DE MONOPROCESADOR.
- EN LOS SISTEMAS DISTRIBUIDOS RESULTA MUY DIFICIL IMPLANTAR ALGORITMOS PARA EVITAR LOS BLOQUEOS:
 - ◆ SE REQUIERE SABER DE ANTEMANO LA PROPORCION DE CADA RECURSO QUE NECESITARA CADA PROCESO:
 - ⇒ ES MUY DIFICIL DISPONER DE ESTA INFORMACION EN FORMA PRACTICA.
- LAS TECNICAS MAS APLICABLES PARA EL ANALISIS DE LOS BLOQUEOS EN SISTEMAS DISTRIBUIDOS SON:
 - ◆ DETECCION.
 - ◆ PREVENCIÓN.

DETECCION DISTRIBUIDA DE BLOQUEOS

- DETECCION CENTRALIZADA DE BLOQUEOS.
- CADA MAQUINA MANTIENE LA GRAFICA DE RECURSOS DE SUS PROPIOS PROCESOS Y RECURSOS.
- UN COORDINADOR CENTRAL MANTIENE LA GRAFICA DE RECURSOS DE TODO EL SISTEMA:
 - ◆ ES LA UNION DE TODAS LAS GRAFICAS INDIVIDUALES.
- CUANDO EL COORDINADOR DETECTA UN CICLO ELIMINA UNO DE LOS PROCESOS PARA ROMPER EL BLOQUEO.
- LA INFORMACION DE CONTROL SE DEBE TRANSMITIR EXPLICITAMENTE, EXISTIENDO LAS SIGUIENTES VARIANTES:
 - ◆ CADA MAQUINA INFORMA CADA ACTUALIZACION AL COORDINADOR.
 - ◆ CADA MAQUINA INFORMA PERIODICAMENTE LAS MODIFICACIONES DESDE LA ULTIMA ACTUALIZACION.
 - ◆ EL COORDINADOR REQUIERE LA INFORMACION CUANDO LA NECESITA.

DETECCION DISTRIBUIDA DE BLOQUEOS

- LA INFORMACION DE CONTROL INCOMPLETA O RETRASADA PUEDE LLEVAR A FALSOS BLOQUEOS:
 - ◆ EL **COORDINADOR** INTERPRETA ERRONEAMENTE QUE EXISTE UN BLOQUEO Y ELIMINA UN PROCESO.
 - ◆ UNA POSIBLE SOLUCION ES UTILIZAR EL **ALGORITMO DE LAMPORT** PARA DISPONER DE UN **TIEMPO GLOBAL**.

DETECCION DISTRIBUIDA DE BLOQUEOS

- EL MENSAJE SE ENVIA AL PROCESO DEBIDO AL CUAL SE BLOQUEA:
 - SI SE BLOQUEA DEBIDO A VARIOS PROCESOS LES ENVIA MENSAJES (DIFERENTES) A TODOS ELLOS.
- ◆ SI UN MENSAJE RECORRE TODO EL CAMINO Y REGRESA A SU EMISOR ORIGINAL (EL PROCESO ENLISTADO EN EL PRIMER CAMPO), ENTONCES:
 - EXISTE UN CICLO Y EL SISTEMA ESTA BLOQUEADO.
- UNA FORMA DE **ROMPER EL BLOQUEO** ES QUE EL PROCESO QUE INICIO LA EXPLORACION SE COMPROMETA A SUICIDARSE:
 - ◆ SI VARIOS PROCESOS SE BLOQUEAN AL MISMO TIEMPO E INICIAN EXPLORACIONES, TODOS ELLOS SE SUICIDARAN.
- UNA VARIANTE ES ELIMINAR SOLO AL PROCESO DEL CICLO QUE TIENE EL N° MAS ALTO.

DETECCION DISTRIBUIDA DE BLOQUEOS

- **DETECCION DISTRIBUIDA DE BLOQUEOS.**
- UN ALGORITMO TIPICO ES EL DE **CHANDY-MISRA-HAAS**.
- LOS PROCESOS PUEDEN **SOLICITAR VARIOS RECURSOS** (POR EJEMPLO CERRADURAS) AL MISMO TIEMPO, EN VEZ DE UNO CADA VEZ.
- SE PERMITEN LAS **SOLICITUDES SIMULTANEAS** DE VARIOS PROCESOS:
 - ◆ UN PROCESO PUEDE ESPERAR A UNO O MAS RECURSOS SIMULTANEAMENTE.
 - ◆ LOS RECURSOS QUE ESPERA UN PROCESO PUEDEN SER **LOCALES O REMOTOS** (DE OTRA MAQUINA).

PREVENCION DISTRIBUIDA DE BLOQUEOS

DETECCION DISTRIBUIDA DE BLOQUEOS

- SI EL PROCESO "0" SE BLOQUEA DEBIDO AL PROCESO "1":
 - ◆ SE GENERA UN MENSAJE DE **EXPLORACION** QUE SE ENVIA AL PROCESO (O PROCESOS) QUE DETIENEN LOS RECURSOS NECESARIOS.
 - ◆ EL MENSAJE CONSTA DE TRES NUMEROS:
 - EL PROCESO RECIEN BLOQUEADO, EL PROCESO QUE ENVIA EL MENSAJE Y EL PROCESO AL CUAL SE ENVIA.
 - ◆ AL LLEGAR EL MENSAJE EL **RECEPTOR VERIFICA** SI EL MISMO ESPERA A ALGUNOS PROCESOS, EN CUYO CASO:
 - EL MENSAJE SE ACTUALIZA:
 - SE CONSERVA EL PRIMER CAMPO.
 - SE REEMPLAZA EL SEGUNDO POR SU PROPIO N° DE PROCESO Y EL TERCERO POR EL N° DEL PROCESO AL CUAL ESPERA.

PREVENCION DISTRIBUIDA DE BLOQUEOS

- LA PREVENCION CONSISTE EN EL **DISEÑO CUIDADOSO** DEL SISTEMA PARA QUE LOS BLOQUEOS SEAN **IMPOSIBLES** ESTRUCTURALMENTE.
- ENTRE LAS **DISTINTAS TECNICAS** SE INCLUYE:
 - ◆ **PERMITIR** A LOS PROCESOS QUE SOLO CONSERVEN UN RECURSO A LA VEZ.
 - ◆ **EXIGIR** A LOS PROCESOS QUE SOLICITEN TODOS SUS RECURSOS DESDE UN PRINCIPIO.
 - ◆ HACER QUE **TODOS LOS PROCESOS LIBEREN** TODOS SUS RECURSOS CUANDO SOLICITEN UNO NUEVO.
- EN UN SISTEMA DISTRIBUIDO CON **TIEMPO GLOBAL** Y **TRANSACCIONES ATOMICAS**:
 - ◆ SE PUEDE **ASOCIAR** A CADA TRANSACCION UNA MARCA DE **TIEMPO GLOBAL** AL MOMENTO DE SU INICIO.
 - ◆ **NO PUEDEN HABER PAREJAS** DE TRANSACCIONES CON IGUAL MARCA DE TIEMPO ASOCIADA.

PREVENCION DISTRIBUIDA DE BLOQUEOS

- LA IDEA ES QUE CUANDO UN **PROCESO** ESTA A PUNTO DE **BLOQUEARSE** EN ESPERA DE UN **RECURSO** QUE ESTA UTILIZANDO OTRO PROCESO:
 - ◆ SE VERIFICA CUAL DE ELLOS TIENE LA MARCA DE TIEMPO MAYOR (ES MAS JOVEN).
 - ◆ SE PUEDE **PERMITIR LA ESPERA** SOLO SI EL PROCESO EN ESTADO DE ESPERA TIENE UNA MARCA INFERIOR (MAS VIEJO) QUE EL OTRO.
- AL SEGUIR CUALQUIER CADENA DE PROCESOS EN ESPERA:
 - ◆ LAS MARCAS APARECEN EN **FORMA CRECIENTE**.
 - ◆ LOS CICLOS SON IMPOSIBLES.

PREVENCION DISTRIBUIDA DE BLOQUEOS

- **OTRA POSIBILIDAD** ES PERMITIR LA ESPERA DE PROCESOS SOLO SI EL **PROCESO QUE ESPERA** TIENE UNA MARCHA MAYOR (ES MAS JOVEN) QUE EL OTRO PROCESO:
 - ◆ LAS MARCAS APARECEN EN LA CADENA EN **FORMA DESCENDENTE**.
- ES MAS SABIO DAR **PRIORIDAD A LOS PROCESOS MAS VIEJOS**:
 - ◆ SE HA INVERTIDO TIEMPO DE PROCESO EN ELLOS.
 - ◆ PROBABLEMENTE CONSERVAN MAS RECURSOS.